

Subutai – WTG Internal Use Only

1. Introduction	2
2. Name Origin – SubutAI	3
3. Problem Definition	3
4. The SubutAI Approach.....	3
4.1 Structured Incident Analysis Framework.....	4
4.2 Evidence-Driven Decision Engine	4
4.3 Intelligent Data Processing (eDocs + Screenshots).....	5
4.4 Historical Intelligence & Pattern Recognition.....	6
4.5 Context-Aware Reasoning	6
4.6 Resolution Intelligence (Triage Filtering).....	6
4.7 Anti-Hallucination Framework.....	6
4.8 Customer-Ready Response Engine	7
4.9 Confidence-Based Decision System	7
4.10 High Performance Execution	7
4.11 Integrated Learning System	8
4.12 Real-Time Incident Correlation & Early Signal Detection	8
5. Alignment with WiseTech AI Strategy	8
6. Delivered Value.....	9
7. Why SubutAI is an Innovation.....	9
8. Conclusion.....	9
9. How to Use?.....	10
10. Sample Investigation	11

SubutAI – AI-Powered Incident Analysis and Decision System

1. Introduction

SubutAI is an advanced artificial intelligence solution developed to redefine technical incident management within the WiseTech Global ecosystem. Unlike traditional support systems, SubutAI does not merely generate responses; it operates as an intelligent incident management engine that orchestrates data analysis, decision-making, and solution generation end-to-end.

The primary objectives of SubutAI are to deliver faster resolutions, enable more accurate decision-making, standardize support processes, and increase trust in artificial intelligence.

```
7  ## 1) Purpose
8
9  Use this instruction set to investigate CargoWise incidents with the
10 existing 10-section analysis format, while producing stronger
11 client-facing responses in Section 8.
12
13 Goals:
14 - Keep current investigation depth and speed (FAST/Detailed).
15 - Improve customer response accuracy and safety.
16 - Reduce unnecessary evidence requests.
17 - Reduce hallucinated UI guidance and invalid field instructions.
18
19 ## 2) Response Modes
20 - Default Mode: FAST
21 - Detailed Mode Trigger: Only when explicitly requested (detayli, deep,
22 full analysis, comprehensive)
23 - FAST Output Scope: Sections 1-8
24 - Detailed Output Scope: Sections 1-10
25 - Language Enforcement: Always respond in English, even if the user
26 writes in another language.
27 - Do not switch output language unless the user explicitly asks for
28 non-English output.
```

2. Name Origin – SubutAI

The name SubutAI is derived from Subutay, one of the most successful military strategists in history. Subutay was renowned for analyzing complex environments, integrating multiple sources of intelligence, and making rapid yet highly accurate decisions under uncertainty. He is widely regarded as one of the few commanders in history to have maintained an undefeated record in battle.

SubutAI brings these same principles into the digital domain by analyzing complex incident data, integrating multiple data sources such as eDocs, historical incidents, and system-level information, and determining the most accurate resolution path while minimizing risk.

For this reason, SubutAI should not be viewed as a simple chatbot, but rather as a digital strategy engine.

3. Problem Definition

Modern support operations face several critical challenges, including repeated analysis of similar incidents, incorrect guidance caused by incomplete data, unnecessary customer data requests, inconsistent resolution quality, extended resolution times, and a lack of trust in AI-generated outcomes.

These challenges significantly reduce operational efficiency and negatively impact customer experience.

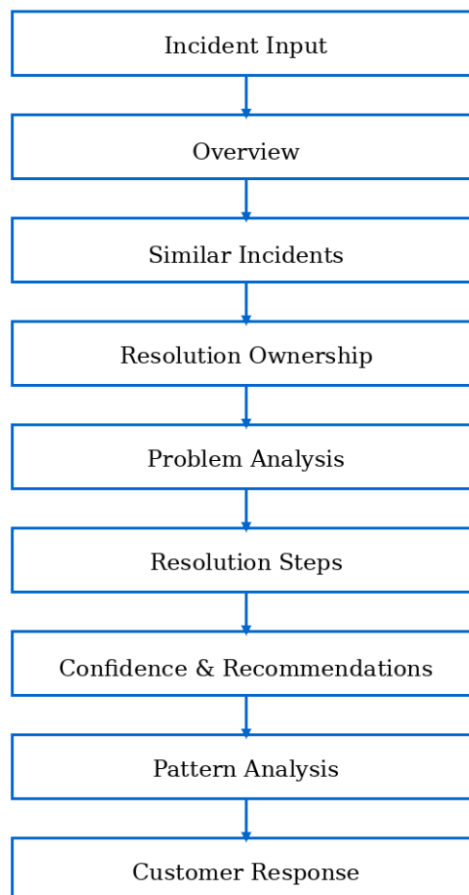
4. The SubutAI Approach

SubutAI fundamentally re-engineers the incident resolution lifecycle by introducing a structured, evidence-driven, and context-aware analysis model.

1. Incident Overview & Investigation	
Field	Details
Incident No	CSC
Issue Description	"Self-Managed OAuth Identity" checkbox not visible under EDI Client Details → Inbound tab; customer asking how to enable it
Customer / Context	Jaç ns Private Limited). Inbound eAdaptor Next setup query
Product / Module	ENT / EAN (eAdaptor Next)
Current Status	CSV — CR5 (Training Question). Triage completed by Indera Ramjiawan; no investigation task started yet
Missing Evidence	Hosting model: WiseCloud vs Self-Hosted — this is the single key variable controlling feature visibility

4.1 Structured Incident Analysis Framework

Each incident is analyzed across a standardized 10-step framework including Incident Overview, Similar Incidents, Resolution Ownership, Problem Analysis, Resolution Steps, Confidence & Recommendations, Solution Pattern Analysis, Customer Response, Timeline & Risk, and Validation Metrics. This ensures consistency, repeatability, and comparability across all incidents.



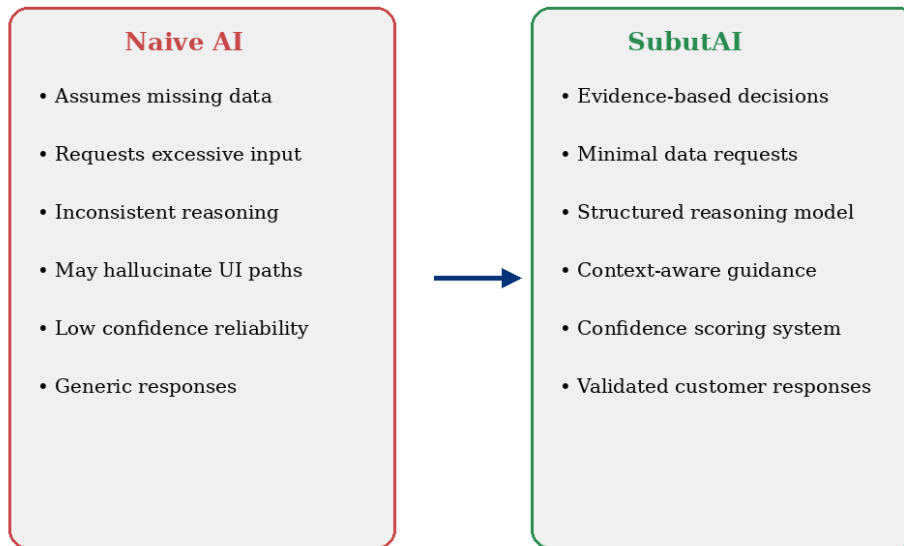
4.2 Evidence-Driven Decision Engine

SubutAI operates strictly on validated evidence. It analyzes all available data sources including messages, eDocs, and screenshots, identifies missing information, and requests only the minimum necessary additional data.

This approach reduces unnecessary communication while significantly improving accuracy.

Naive AI vs SubutAI

Evidence-Driven Decision Model Comparison



4.3 Intelligent Data Processing (eDocs + Screenshots)

SubutAI intelligently processes incident data by prioritizing relevant sources. Text-based files such as emails and logs are analyzed first, while heavier files such as images or compressed data are only processed when required.

This ensures optimal performance while maintaining analytical depth.

 **4. Problem Analysis**

Root cause: The "Self-Managed OAuth Identity" checkbox is a self-hosted-exclusive feature, hidden by design for WiseCloud customers. Per product documentation (*How-to Setup EDI Client Details v1.9*): "If you are Self-Hosted, you can see a 'Self-Managed OAuth Identity' checkbox." No equivalent toggle exists for WiseCloud; WiseCloud inbound OAuth uses the WiseTech-managed Entra ID provisioning path (CSR upload → generate certificate).

Hosting model is the single determinant. It cannot be confirmed from incident metadata or system data available. This resolves the case given one answer:

- WiseCloud → option intentionally absent; use standard Entra ID inbound OAuth path
- Self-Hosted + still not visible → security rights gap or eAdaptorNext module not enabled in the environment
- Self-Hosted + visible → customer can tick the checkbox and supply their own Authorization Server URL and Client ID; only Entra ID is supported as IdP

4.4 Historical Intelligence & Pattern Recognition

SubutAI leverages historical incidents to identify patterns, match similar cases, and accelerate resolution for recurring issues and arranges them chronologically. If the incident is classified as a “Defect”, the corresponding work item reference is displayed.

2. Similar Incidents History

CS02 (Jul 2025) — eAdaptor Next inbound OAuth setup; customer expected a Client Secret for inbound — resolved by directing to EDI Client certificate-based auth guide

CS02 (Oct 2025) — SOAP/eAdaptor Next migration query; agent explicitly confirmed Self-Managed OAuth Identity is only visible for self-hosted customers, with Entra ID as the only supported IdP

CS02 (Nov 2025) — OAuth Authority settings not visible in registry (self-hosted CWN); resolved by explaining those settings are legacy; self-hosted customers must enable eAdaptorNext to unlock EDI Client Details module for OAuth setup

4.5 Context-Aware Reasoning

SubutAI distinguishes between different operational entities such as Shipment, Consol, and Declaration, ensuring that solutions are always generated within the correct context.

4.6 Resolution Intelligence (Triage Filtering)

SubutAI excludes triage tasks from resolution analysis and identifies the actual contributors responsible for resolving incidents. This enables rapid identification of the right expertise when needed.

3. Resolution Team & Personnel Summary

Top Teams (max 3):

Team / Department	Incidents Resolved	Common Approach
eAdaptor Support (EDS)	3	Directed customers to How-to Setup EDI Client Details eLearning; confirmed hosting-dependent feature availability
eAdaptor Product Management (EDP)	1	Assisted with eAdaptorNext enablement for self-hosted customers

Top Personnel (max 3):

Name	Department	Incidents Resolved
Amberly Munoz	EDS	1
Mpho Chiloane	EDS	1
Audric Pappalardo	EDS / EDP	1

4.7 Anti-Hallucination Framework

SubutAI prevents hallucination by avoiding unverified assumptions, requiring evidence-based validation, and requesting clarification when uncertainty exists. This ensures enterprise-grade reliability.

4.8 Customer-Ready Response Engine

SubutAI generates professional, clear, and actionable responses that are immediately usable and aligned with corporate communication standards.

✓ 5. Resolution Steps

A. Confirm hosting model from the customer (single targeted question in Section 8).

B. If **WiseCloud**: close as training — explain the option is not available by design; guide to the standard Entra ID certificate-based inbound OAuth path via EDI Client Details.

C. If **Self-Hosted** and option is absent: request a screenshot of the Inbound tab on their EDI Client Details record; verify the user has the *EDI Client Details* security right assigned; confirm eAdaptorNext is activated in their environment.

D. If **Self-Hosted** and option is present: guide the customer to tick "Self-Managed OAuth Identity" which will reveal fields for Authority URL and Client ID (their own Authorization Server). Remind that only Entra ID is supported as identity provider.

Thank you for your eRequest regarding the Self-Managed OAuth Identity option.

The "Self-Managed OAuth Identity" checkbox in the EDI Client Details Inbound section is only available for self-hosted CargoWise customers. As your environment is WiseCloud-hosted, this option is intentionally not displayed — this is expected product behaviour, not a configuration issue.

The good news is that your inbound OAuth is already fully set up. Looking at your "Flow Forwarding" EDI Client record, the following values are already populated:

- **OAuth Authority URL** — your WiseTech-managed Entra ID token endpoint
- **OAuth Client ID** — your unique application identifier
- **Scope** — auto-populated for authentication

These are the credentials your integration or middleware should use to obtain a bearer token and authenticate inbound requests to eAdaptor Next. You do not need the Self-Managed Identity option to proceed.

For guidance on using these credentials in Postman or configuring your middleware, please refer to Section 2.3 and 2.5 of the eAdaptor Next Developer's Guide: <https://wisetechacademy.com/search?quickstart=a1bb2e18-1e7d-45d8-9969-e4182d1dfd9e>

Please let us know if you have any further questions.

Thank You,
Enes

4.9 Confidence-Based Decision System

Each resolution includes a confidence score calculated based on data completeness, similarity to historical incidents, and clarity of root cause, enhancing transparency and trust.

💡 6. Confidence & Recommendations

Confidence: 72% — Root cause is well understood and supported by documentation and multiple historical resolutions. Only the hosting model remains unconfirmed. A single client reply is expected to resolve this incident. No product defect signals present.

Recommendation: Respond with the targeted hosting model question (Section 8) and pre-load both outcome paths. Likely Training resolution once hosting model is confirmed.

4.10 High Performance Execution

SubutAI is optimized for performance, delivering results in approximately 30–50 seconds in FAST mode and 2–4 minutes in Detailed mode. It also shows the current percentage of the process to avoid curiosity.

Executed parallel queries for job details and incidents

 Analysing... 10%

4.11 Integrated Learning System

SubutAI continuously improves by recommending relevant learning materials, educating users, and reducing recurring issues over time.

Relevant eLearning:

- eAdaptor Next Developer's Guide & Sample Interfaces — <https://wisetechacademy.com/search?quickstart=a1bb2e18-1e7d-45d8-9969-e4182d1dfd9e>
- Enabling Open ID Connect as an authentication option for self-hosted customers — <https://wisetechacademy.com/search?quickstart=17b9a9c3-f4f2-421b-b75c-42ca334be383>
- How-to Troubleshoot eAdaptor Next Integration — <https://wisetechacademy.com/search?quickstart=42cca1d0-315e-4325-ad2b-9d8980804fe6>

4.12 Real-Time Incident Correlation & Early Signal Detection

SubutAI continuously analyzes newly created incidents within recent time windows, with particular emphasis on the last 24 hours and the most recent 1–2 hour period. By correlating incidents across multiple organizations and detecting high-similarity patterns, the system can identify emerging issues at an early stage.

This capability enables SubutAI to distinguish between user-specific problems and system-wide anomalies in near real time. As a result, potential platform-level issues can be recognized before they escalate, allowing for faster escalation, proactive response, and significantly improved incident resolution efficiency.

Recent Similar Incident Check (24h / 1–2h):

CS02287683 (today, outbound communication guide, EAN) — similarity ~40% to this incident; excludes from pattern signal.

No incidents with ≥70% similarity in the last 24 hours.

Signal: User-Specific — Single organisation, isolated query, no cross-organisation cluster indicating a system-wide issue.

5. Alignment with WiseTech AI Strategy

SubutAI aligns with WiseTech's AI vision by improving operational efficiency, augmenting decision-making, enabling scalability, and ensuring reliability.

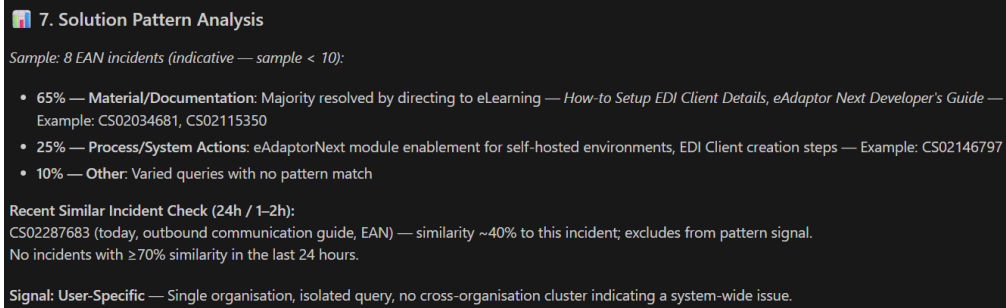


6. Delivered Value

SubutAI reduces MTTR, accelerates incident resolution, minimizes repetitive work, improves customer satisfaction, and reduces error rates.

7. Why SubutAI is an Innovation

SubutAI is not a simple chatbot or an LLM wrapper. It is a combination of structured reasoning, evidence-driven intelligence, pattern recognition, and safety-first architecture, delivering a controllable, auditable, and reliable AI system.



8. Conclusion

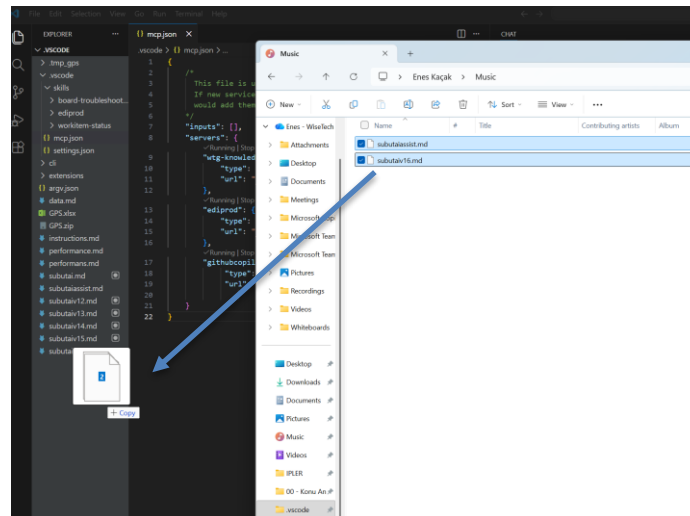
SubutAI transforms technical support operations by enabling strategic analysis, data-driven decision-making, risk minimization, and improved customer experience. It represents the foundation of next-generation AI-powered support systems.

9. How to Use?

SubutAI is a drag-and-drop based prompt framework designed for seamless integration into development environments. The subutaivXX.md file and subutaiassistvXX file can be easily added to Visual Studio Code by dragging it into the Explorer panel.

You can download relevant files and find more informations from; <https://ai.wtg.zone/tool/subutai> and <https://github.com/enesrazor/subutai>

Step 1: Please be sure that all of URLs are working under the mcp.json file. Drag and drop the relevant subutaivXX.md and subutaiassistvXX.md file into the Explorer panel in Vscode (Or you can simply copy the file under the C:\Users\.....\.vscode folder)



Step 2: Open the file from the Explorer panel and either pin it or drag it into the Chat interface (subutaivxx.md file). Once loaded, you can initiate an analysis by entering a simple prompt such as:

“CSXXXXXX” SubutAI will automatically process the request and return the structured incident analysis.



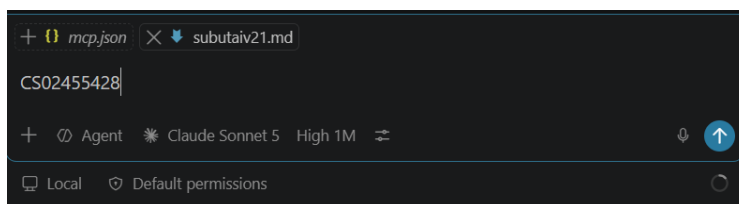
If you are using multiple agents or prefer not to pin or drag the subutaivXX.md file to the Chat interface, you can explicitly reference it within your prompt. For example:

“Investigate CSXXXXXX with the help of the subutaivXX.md file.”

This ensures that SubutAI logic is applied from “Explorer” panel even without direct attachment.

10. Sample Investigation

Step1: Drag subutaiv21.md file to the Chat section and type incident number



Step 2: Investigation has started and you’ll have a reply in 40 seconds (in FAST mode) or 2-3 minutes in detailed mode.



Step 3: Let’s review the results step by step. Once the results are available, we’ll start with Section 1. This section provides a summary of the incoming incident, including the company/person who raised it, information about the person who submitted it, whether the company uses a self-hosted or hosted environment, and other relevant details.

1. Incident Overview & Investigation	
Field	Details
Incident No	CS02455428
Issue Description	Client asks whether there is a way to confirm all runtime dependencies are satisfied <i>before</i> triggering a scheduled CargoWise upgrade. Background: an earlier upgrade failed due to missing .NET Core runtime; after remediating that, a subsequent upgrade attempt failed again — this time due to missing “desktop runtimes.” Client runs upgrades out-of-hours and wants to avoid discovering missing prerequisites mid-window.
Customer / Context	Admin - Rob Saw, Charles Kendall Freight Limited (CHAKENLON), enterprise AIY, GB, self-hosted CargoWise environment
Product / Module	ENT (CargoWise) / PER (Architecture Core) - APP (Application Installations)
Current Status	CSV - Customer Service Actioning; criticality set to CR4; raised 2026-09-07, still in early triage/investigation workflow
Missing Evidence	None required to answer the core question. The actual failed-upgrade log (would only be needed if the goal were to diagnose <i>this specific</i> failure rather than to explain the proactive check mechanism itself)

In Step 2, the history of similar incidents is searched. Past incidents with a similarity score of 70% or higher, along with any recent incidents from the same company with a similarity score of 90% or higher, are reviewed, and matching incidents are displayed.

2. Similar Incidents History

- CS01953574 (2025-05-07) - Failed upgrade to GP1 25.4.7.127 due to missing .NET 8 runtimes on SQL/Citrix servers; support explained the UPG check reads cached `dbo.StmData` (15-day freshness window), not a live scan.
- CS01973981 (2025-05-28) - Self-hosted customer asked which .NET 8 components were needed; support recommended Hosting Bundle + Desktop Runtime, reboot, wait 24h, then relaunch CargoWise/Process Controller to refresh the cache.
- CS02005922 (2025-07-01) - Near-identical request ("check PCs that don't meet the .NET runtime requirement before our scheduled upgrade"). Resolved by Benjamin.Wang with the exact same `SELECT ... FROM dbo.StmData WHERE SD_Name LIKE 'DOTNETCORE[_]RUNTIMES[%]'` query used in this incident, confirming a 15-day refresh cycle. Closed as Training.

No recurring same-issue pattern found for this customer (CHAKENLON) in the last 12 months.

In Step 3, the people who resolved incidents similar to the one being analyzed, along with their teams, are displayed, excluding those who only handled triage tasks. If needed, these people or teams can be contacted promptly for assistance with the issue that led to the incident.

3. Resolution Team & Personnel Summary

Top Teams:

Team / Department	Incidents Resolved	Common Approach
Application Installations / Architecture Core Support	3 (of 3 sampled)	Point customer to the <code>dbo.StmData</code> query and the Desktop Runtime vs. Hosting Bundle distinction rather than waiting for the next failed upgrade attempt

Top Personnel:

Name	Department	Incidents Resolved
Enes Kaçak	Support	1 (this incident)
Benjamin.Wang	Support	1 (CS02005922 — same SQL query precedent)

Sample too small for a statistically meaningful ranking beyond these two precedents.

In Step 4, the issue reported in the incident is analyzed in detail to identify its likely root cause. This section explains the relevant system behavior, evaluates the available evidence and findings from similar incidents, and assesses whether the issue is related to environment configuration, missing prerequisites, or a potential product defect. It also provides the reasoning behind the assessment and indicates whether a code or product fix may be required.

4. Problem Analysis

CargoWise's pre-upgrade check (UPG service) does not scan machines live; it reads a cached per-machine .NET runtime state stored in `dbo.StmData` (key pattern `DOTNETCORE_RUNTIMES[<machine>]`), which is only refreshed when CargoWise/Process Controller is actually launched on that machine, and only if the existing record is more than 15 days old. This explains both of the customer's failures: after the first failure they updated the runtime but the cached record hadn't yet refreshed (or a different component was missing), and the second failure was for a different required component — the .NET Desktop Runtime — which is a separate installer from the ASP.NET Core Hosting Bundle. Installing one does not install the other, and CargoWise now requires .NET 10 versions of both (Hosting Bundle + Desktop Runtime) in addition to the existing .NET 8 / .NET Framework 4.8, per the WTA quickstart referenced. This is a self-service, environment-configuration issue rather than a product defect — no code/product fix is indicated, and it matches a well-established, repeatedly-resolved pattern (7+ precedent incidents).

In Step 5, the resolution steps are presented in a clear, actionable sequence. This section outlines the checks to perform, any required configuration changes or installations, and relevant documentation or queries to support the process. It also explains how to verify the results and clarifies whether further action from Support is required.

5. Resolution Steps

1. Provided the direct SQL query against `dbo.StmData` so the customer can check the currently recorded runtime state per machine without waiting for an upgrade attempt to surface it.
2. Explained the Hosting Bundle vs. Desktop Runtime distinction and which machine types need which component (servers/process controllers → Hosting Bundle; RDS/Citrix hosts and full desktop clients → Desktop Runtime).
3. Directed to the current .NET 10 requirement WTA quickstart and the Microsoft download page for both installers.
4. Advised installing components several days ahead of the maintenance window (not immediately before), launching CargoWise once on each machine to force the cache refresh, and re-running the query a day or two later to confirm the refreshed date before attempting the upgrade.

No further action is required from Support unless the client reports the query result differs from expectation.

In Step 6, a confidence score is provided for the analysis and proposed resolution, along with the reasoning behind it. This section highlights the supporting evidence from similar incidents and any missing information that limits confidence. It also provides recommendations on whether escalation is needed and outlines the next diagnostic steps if the issue persists.

6. Confidence & Recommendations

Confidence: 4/5 — the mechanism (StmData cache, 15-day freshness window) and the exact SQL query are directly corroborated by CS02005922 and CS01953574, and the Hosting Bundle vs. Desktop Runtime distinction matches the pattern seen across 7+ prior incidents. Not 5/5 because there is no incident-specific evidence (e.g. a screenshot of this customer's own StmData query results) confirming which exact machines/components were missing in their environment.

Recommendation: no escalation needed. If the client reports the query still shows stale/missing entries after following the steps, the next diagnostic step is to confirm they actually relaunched CargoWise on each affected machine (the most common miss in this pattern) before assuming a product-side caching defect.

In Step 7, resolution patterns from similar incidents are analyzed and grouped by solution type, with percentage breakdowns and supporting incident references. This section shows how similar issues have been resolved, such as through process/system actions or guidance and documentation.

The “Signal” field assesses whether the issue is “User-Specific” or a “Potential Defect.” If a large number of incidents with similarity scores above 70% have been reported within the last 1–2 hours or the last 24 hours, it flags a possible defect. It also examines the incident's content and recent similar incidents to identify potential defects beyond volume-based signals, helping detect emerging issues early.

7. Solution Pattern Analysis

- 90% - Process/System Actions: Install missing .NET component(s) (Hosting Bundle and/or Desktop Runtime) on affected machines, relaunch CargoWise to refresh the `dbo.StmData` cache, then retry the upgrade — CS01953574, CS01973981, CS02005922, CS02382261.
- 10% - Material/Documentation: Pointing to WTA update notes/eLearning as sufficient without further action needed — CS02005922 (closed as Training).

Signal: User-Specific (environment-configuration/deployment-readiness issue tied to which machines have had CargoWise relaunched recently), not a system-wide defect.

In Step 8, a ready-to-use customer response is provided, bringing together the explanation of the issue, recommended resolution steps, and relevant references in a clear, customer-friendly format. The following section includes the confidence score and any additional actions or follow-up required.

■ 8. Customer Response (Already Sent — 2026-09-10, by Enes Kaçak)

Hi Rob,

Yes, there is a way to check this proactively rather than finding out when the upgrade runs. CargoWise records each machine's installed .NET runtime state (in the database, table dbo.StmData) the first time CargoWise is loaded on that machine each day, and the upgrade only checks records that are no more than 15 days old at the moment it starts. You can query that table directly at any time to see exactly what is currently recorded per machine, without needing to attempt an upgrade first:

```
SELECT REPLACE(SD_Name, 'DOTNETCORE_RUNTIMES', '') AS Machine, CAST(SD_BinaryValue AS varchar(MAX)) AS Value FROM dbo.StmData WHERE SD_Name LIKE 'DOTNETCORE[_]RUNTIMES]%'
```

Based on the pattern of your two failures (missing .NET Core runtime the first time, missing "desktop runtimes" the second), this looks like the same root cause we see in other self-hosted environments: the ASP.NET Core Runtime – Windows Server Hosting Bundle and the .NET Desktop Runtime are two separate downloads from Microsoft, and installing one does not install the other. Servers and process controllers need the Hosting Bundle; Remote Desktop/Citrix hosts and any desktop running the full CargoWise client need the Desktop Runtime specifically. The current requirement, per CargoWise now requires .NET 10 (<https://wisetechacademy.com/search?quickstart=3871f09b-a987-4a72-a379-f2932f47cb67>), is that all machines running CargoWise need both the .NET 10 Hosting Bundle and .NET 10 Desktop Runtime installed, in addition to the .NET 8 and .NET Framework 4.8 versions already in place — both are available from Microsoft at <https://dotnet.microsoft.com/en-us/download/dotnet/10.0/runtime>.

To validate ahead of your next window: install both components on every relevant machine, then open CargoWise on that same machine at least once so the new version is recorded, and re-run the query above a day or two later to confirm the recorded date has refreshed. Doing this several days before the scheduled upgrade (rather than immediately beforehand) avoids the 15-day freshness cut-off catching you out again.

Please try this option and let us know about the results.

Kind regards,

THE BELOW IS FOR INFORMATION ONLY, DO NOT SEND TO CLIENT

Confidence: 4/5

DISCLAIMER FOR WISETECH SUPPORT - This response was generated by an AI agent and may not be correct. Please review before acting.

Similar or related incidents:

- CS02005922 (2025-07-01) - Identical request to check PCs against .NET runtime requirement before upgrade; resolved with the same StmData SQL query, closed as Training.
- CS01953574 (2025-05-07) - Failed upgrade due to missing .NET 8 runtimes; explained StmData caching and 15-day freshness window.
- CS01973981 (2025-05-28) - Self-hosted upgrade failure; clarified Hosting Bundle + Desktop Runtime + 24h cache refresh requirement.
- CS02382261 (2026-06-24) - DB upgrade failed after .NET 10 install due to two machines missing .NET Desktop Runtime 10.0 specifically.
- CS01992954 (2025-06-17) - Self-hosted upgrade aborted for missing .NET 8/Desktop/ASP.NET Core runtimes across multiple listed hosts.

Relevant WiseTech Academy links:

- CargoWise now requires .NET 10 - <https://wisetechacademy.com/search?quickstart=3871f09b-a987-4a72-a379-f2932f47cb67>

Starting: Review evidence and check for gaps (3/4)